

# TRANSFER CREDIT MATCH SYSTEM

*Data Science Branch · Proof of Concept Documentation*

|                  |                                                                                                   |
|------------------|---------------------------------------------------------------------------------------------------|
| Course / Subject | CST 378 — Data Science                                                                            |
| Project Branch   | Data_Science                                                                                      |
| Document Type    | Technical Proof of Concept Documentation                                                          |
| Version          | 1.0 — Initial Release                                                                             |
| Last Updated     | April 2026                                                                                        |
| Team Role        | Data Science / Database Pipeline                                                                  |
| Status           | Active Development — Proof of Concept Phase                                                       |
| Institutions     | Roosevelt University, National Louis University, University of Chicago, Harold Washington College |

### Executive Summary

The Transfer Credit Match (TCM) system automates the evaluation of transfer credits between Chicago-area colleges by mapping courses to standardized Knowledge Units (KUs) aligned with the NSA/DHS Centers of Academic Excellence (CAE) framework. This document provides complete technical documentation of the Data Science branch — covering the data pipeline, PDF transcript parser, KU taxonomy, matching algorithm, confidence scoring, and director review workflow.

## Table of Contents

|                                                        |   |
|--------------------------------------------------------|---|
| Table of Contents .....                                | 1 |
| 1. Project Overview .....                              | 3 |
| 1.1 Background and Problem Statement .....             | 3 |
| 1.2 Project Objectives.....                            | 3 |
| 1.3 Participating Institutions — Proof of Concept..... | 4 |

|     |                                                     |    |
|-----|-----------------------------------------------------|----|
| 1.4 | Scope Boundaries .....                              | 4  |
| 2.  | System Architecture .....                           | 5  |
| 2.1 | Architectural Overview .....                        | 5  |
| 2.2 | Layer Descriptions .....                            | 5  |
|     | Layer 1 — PDF Transcript Parser .....               | 5  |
|     | Layer 2 — Data Standardization and Validation ..... | 5  |
|     | Layer 3 — Knowledge Unit Matching Engine .....      | 5  |
|     | Layer 4 — Director Review Workflow .....            | 6  |
| 2.3 | Folder Structure.....                               | 6  |
| 3.  | Data Model.....                                     | 6  |
| 3.1 | Entity Overview .....                               | 6  |
| 3.2 | Academic Programs.....                              | 7  |
| 3.3 | Course Catalog — Full 20-Course Sample .....        | 7  |
| 3.4 | Knowledge Unit Taxonomy — 26 CAE-Aligned KUs .....  | 8  |
| 4.  | Knowledge Unit Matching Engine .....                | 9  |
| 4.1 | Matching Algorithm Design .....                     | 9  |
| 4.2 | Confidence Score Thresholds .....                   | 10 |
| 4.3 | Course-to-KU Mapping — Full Seeded Sample .....     | 10 |
| 4.4 | Matching Limitations and Future Improvements.....   | 11 |
| 5.  | PDF Transcript Parser .....                         | 11 |
| 5.1 | Purpose and Motivation .....                        | 11 |
| 5.2 | Technical Approach .....                            | 12 |
| 5.3 | Extracted Fields and Output Schema .....            | 12 |
| 5.4 | Libraries and Dependencies .....                    | 13 |
| 5.5 | Usage .....                                         | 13 |
| 6.  | Data Standardization and Validation Layer .....     | 13 |
| 6.1 | Purpose.....                                        | 13 |
| 6.2 | Standardization Pipeline.....                       | 13 |
| 6.3 | KU Normalization — Mapping Variant Labels .....     | 14 |
| 6.4 | Parser Output Validation .....                      | 14 |
| 6.5 | Benchmark Dataset Generation .....                  | 15 |
| 7.  | End-to-End Transfer Request Example.....            | 15 |
| 7.1 | Scenario Description.....                           | 15 |
| 7.2 | Student and Enrollment Context.....                 | 15 |
| 7.3 | Transfer Request Details .....                      | 16 |
| 7.4 | Matching Engine Processing.....                     | 16 |

|                                              |    |
|----------------------------------------------|----|
| 8. Director Review Workflow .....            | 17 |
| 8.1 Overview .....                           | 17 |
| 8.2 Review State Machine .....               | 17 |
| 8.3 What Directors See.....                  | 17 |
| 8.4 Audit Trail and Compliance .....         | 17 |
| 9. Future Enhancements .....                 | 18 |
| 9.1 Phase 2 Roadmap .....                    | 18 |
| 9.2 Scalability Considerations.....          | 18 |
| 10. Contributors — Data Science Branch ..... | 19 |
| 10.1 Team Contributions .....                | 19 |
| 11. Glossary .....                           | 19 |

# 1. Project Overview

---

## 1.1 Background and Problem Statement

Each year, thousands of students transfer between colleges and universities across the Chicago metropolitan area. A persistent challenge these students face is the ambiguity and inconsistency of the transfer credit evaluation process. Academic advisors at receiving institutions manually compare course syllabi, credit hours, and institutional reputation — a process that is slow, subjective, and difficult for students to navigate or predict.

The Transfer Credit Match (TCM) system addresses this challenge by introducing a structured, data-driven approach to course equivalency evaluation. Rather than relying on advisor judgment alone, TCM compares courses using a shared vocabulary of academic competencies known as Knowledge Units (KUs), enabling consistent and transparent matching across institutions.

## 1.2 Project Objectives

The TCM system has been designed to achieve the following objectives:

- Enable intelligent, semantic course-to-course matching across all participating institutions using a shared KU taxonomy.
- Reduce the manual workload on academic advisors and program directors by automating the preliminary matching step.
- Give transfer students transparent, traceable decisions with clear rationale based on KU overlap evidence.

- Build a fully automated pipeline from PDF transcript ingestion through match approval and audit logging.
- Align the system's academic vocabulary with the CAE (Centers of Academic Excellence) Knowledge Unit framework recognized by the NSA and DHS.
- Create a scalable foundation that can expand beyond the initial four institutions in the proof of concept.

### 1.3 Participating Institutions — Proof of Concept

The proof of concept seeds data from four real Chicago-area academic institutions representing a range of program offerings across computer science, cybersecurity, information technology, and related disciplines:

| ID | Institution               | City        | Programs Represented                                             | Role                 |
|----|---------------------------|-------------|------------------------------------------------------------------|----------------------|
| 1  | Roosevelt University      | Chicago, IL | Computer Science, Cybersecurity, Cyber Security & Info Assurance | Source / Destination |
| 2  | National Louis University | Chicago, IL | Computer Forensics, Computer Science                             | Source / Destination |
| 3  | University of Chicago     | Chicago, IL | Cloud Computing, Information Technology                          | Source / Destination |
| 4  | Harold Washington College | Chicago, IL | Information Technology, Software Development                     | Source / Destination |

*Table 1.1 — All four institutions serve as both source and destination in the POC, meaning a student can transfer in either direction between any pair.*

### 1.4 Scope Boundaries

The current proof of concept is intentionally scoped to validate the core matching logic before scaling. The following items are in and out of scope for this phase:

| Area             | In Scope                             | Out of Scope                               |
|------------------|--------------------------------------|--------------------------------------------|
| Institutions     | 4 Chicago-area colleges              | Illinois statewide / national institutions |
| Courses          | 25 seeded courses across 4 schools   | Full institutional course catalogs         |
| Transcript Input | Banner PDF and web-saved PDF formats | Scanned/image-only PDFs (OCR)              |
| Matching         | KU overlap confidence scoring        | AI/ML-based semantic matching              |
| Review UI        | Director workflow data model         | Web-based director dashboard (planned)     |

| Area     | In Scope                    | Out of Scope                      |
|----------|-----------------------------|-----------------------------------|
| Students | Seeded test student records | Live student authentication / SSO |

## 2. System Architecture

### 2.1 Architectural Overview

The Data Science branch is responsible for the full data pipeline — from raw student transcript ingestion through structured match records ready for director review. The system is built as a sequential processing pipeline with four primary layers, each with clearly defined inputs, outputs, and responsibilities.

#### End-to-End Data Pipeline

[ Student PDF Transcript ]    ↓ Layer 1 [ pdf\_reader.py — Parse courses, grades, credits from transcript ]    ↓ Layer 2 [ Standardization & Validation — Normalize KU terms, flag missing fields ]  
↓ [ Transfer Request submitted to Database ]    ↓ Layer 3 [ KU Matching Engine — Compute confidence score via Jaccard similarity ]    ↓ [ course\_match record created — Status: Pending ]  
↓ Layer 4 [ Director Review — Approve / Deny / Escalate ]    ↓ [ match\_history — Audit trail preserved ]

### 2.2 Layer Descriptions

#### Layer 1 — PDF Transcript Parser

The entry point of the pipeline. Accepts student academic transcripts in Banner-generated or web-saved PDF formats and extracts structured course records. Outputs a structured dataset of course codes, course names, grades, and credit hours that feeds into the transfer request submission flow. Handles layout noise common in web-printed transcripts through regex-based fallback parsing.

#### Layer 2 — Data Standardization and Validation

Validates and normalizes parser output before it enters the matching pipeline. This layer checks for missing required fields (course code, title, credit hours), maps variant KU terminology into a canonical internal naming scheme, and generates synthetic benchmark datasets for testing purposes. Rows failing validation are flagged for manual review rather than silently dropped.

#### Layer 3 — Knowledge Unit Matching Engine

The core intelligence of the system. For each transfer request, the engine retrieves the KU profiles of both the source course (completed by the student) and the target course (at the destination institution), then computes a Jaccard similarity-based confidence score. The score drives automated thresholding: high-

confidence matches are flagged for quick director confirmation, medium-confidence matches trigger full manual review, and low-confidence matches are rejected.

#### Layer 4 — Director Review Workflow

The human-in-the-loop layer. Program directors at destination institutions review pending match records alongside KU overlap evidence. Directors may approve, deny, or escalate each match. All decisions are timestamped and written to the `match_history` audit log, providing a permanent, auditable record of every transfer credit decision.

## 2.3 Folder Structure

The Data Science branch is organized into two primary subdirectories: the transcript parser and the CSV data exports used to seed the database:

```
Data_Science/
├── readme.txt                                ← Branch overview (this document's source)
├── TranscriptPDF_reader/
│   └── pdf_reader.py                        ← PDF transcript parsing script
├── csv_exports/
│   ├── institutions.csv                    ← 4 participating Chicago-area colleges
│   ├── programs.csv                      ← 9 academic programs across institutions
│   ├── courses.csv                       ← 25 seeded courses with codes and credits
│   ├── knowledge_units.csv               ← 26 CAE-aligned KU definitions
│   ├── course_ku.csv                     ← 32 course-to-KU links (many-to-many)
│   ├── students.csv                     ← Seeded student enrollment records
│   ├── transfer_requests.csv             ← Student-submitted transfer requests
│   ├── course_match.csv                  ← Matched course pairs with status
│   ├── match_history.csv                 ← Immutable audit log of all decisions
│   ├── admins.csv                       ← System administrator records
│   ├── directors.csv                    ← Program director records
│   └── users.csv                         ← Unified user table (all roles)
```

## 3. Data Model

### 3.1 Entity Overview

The TCM data model consists of eleven core entities organized into three logical groups: academic structure (institutions, programs, courses), knowledge taxonomy (knowledge units, course-KU mappings), and operational records (students, transfer requests, course matches, match history, users, admins, directors). The entity relationships are summarized below:

**Key Relationships**

Institution → (has many) Programs → (has many) Courses Course ↔ (many-to-many via course\_ku) ↔ KnowledgeUnit Student → (submits) TransferRequest → (generates) CourseMatch → (logged in) MatchHistory Director → (reviews and decides on) CourseMatch

### 3.2 Academic Programs

Each institution offers one or more programs that students are enrolled in. Program affiliation determines which courses are available to a student and which director is responsible for reviewing incoming transfer requests:

| ID | Institution               | Program Name                    | Domain               |
|----|---------------------------|---------------------------------|----------------------|
| 1  | Roosevelt University      | Computer Science                | CS / Software        |
| 2  | Roosevelt University      | Cybersecurity                   | Security             |
| 3  | Harold Washington College | Information Technology          | IT / Systems         |
| 4  | Harold Washington College | Software Development            | CS / Software        |
| 5  | National Louis University | Computer Forensics              | Security / Forensics |
| 6  | National Louis University | Computer Science                | CS / Software        |
| 7  | University of Chicago     | Cloud Computing                 | Infrastructure       |
| 8  | University of Chicago     | Information Technology          | IT / Systems         |
| 9  | Roosevelt University      | Cyber Security & Info Assurance | Security             |

### 3.3 Course Catalog — Full 20-Course Sample

All courses in the POC are tied to a specific institution and program. The catalog below represents the seeded course data. Five additional courses are seeded beyond this table for a total of 25:

| ID | Code     | Course Name                       | Credits | Institution               |
|----|----------|-----------------------------------|---------|---------------------------|
| 1  | CS101    | Data Structures and Algorithms    | 4       | Roosevelt University      |
| 2  | CYB201   | Network Security                  | 3       | Roosevelt University      |
| 3  | IT101    | IT Fundamentals                   | 3       | Harold Washington College |
| 4  | SD102    | Web Development                   | 3       | Harold Washington College |
| 5  | CSIA 150 | Computer Science I                | 4       | Harold Washington College |
| 6  | CSIA 217 | Intro to Probability & Statistics | 3       | Harold Washington College |
| 7  | CSIA 236 | Python Script Programming         | 3       | Harold Washington College |

| ID | Code     | Course Name                            | Credits | Institution               |
|----|----------|----------------------------------------|---------|---------------------------|
| 8  | CSIA 246 | Data Communications                    | 3       | Harold Washington College |
| 9  | CSIA 250 | Computer Science II                    | 4       | Harold Washington College |
| 10 | CSIA 255 | Open Source Communities                | 3       | Harold Washington College |
| 11 | CSIA 261 | Computer Organization                  | 3       | Harold Washington College |
| 12 | CSIA 301 | Computer Networking                    | 3       | Harold Washington College |
| 13 | CSIA 317 | Operating Systems                      | 3       | Harold Washington College |
| 14 | CSIA 318 | UNIX and System Administration         | 3       | Harold Washington College |
| 15 | CSIA 319 | Cyber Ops                              | 3       | Harold Washington College |
| 16 | CSIA 327 | Software Engineering                   | 3       | Harold Washington College |
| 17 | CSIA 333 | Database Systems                       | 3       | Harold Washington College |
| 18 | CSIA 335 | Ethical Hack & Countermeasures         | 3       | Harold Washington College |
| 19 | CSIA 336 | Practical Computing with Data / Python | 3       | Harold Washington College |
| 20 | CSIA 352 | Network Design                         | 3       | Harold Washington College |

### 3.4 Knowledge Unit Taxonomy — 26 CAE-Aligned KUs

Knowledge Units (KUs) are the semantic backbone of the matching system. Each KU represents a discrete, standardized academic competency area. The 26 KUs seeded in the POC are aligned to the NSA/DHS Centers of Academic Excellence (CAE) curriculum framework, ensuring compatibility with nationally recognized cybersecurity and IT program standards:

| KU ID | Knowledge Unit Name             | Core Competency Description                                              |
|-------|---------------------------------|--------------------------------------------------------------------------|
| 1     | Programming Concepts            | Fundamental logic, control flow, and syntax across programming paradigms |
| 2     | Cybersecurity Basics            | Introductory concepts in information security, threats, and defenses     |
| 3     | Networking Fundamentals         | Basic networking architecture, protocols, and connectivity principles    |
| 4     | Database Management             | Relational and non-relational database design using SQL and NoSQL        |
| 5     | Cybersecurity Foundations       | Core principles underpinning information assurance and cyber defense     |
| 6     | Cybersecurity Principles        | Security design patterns, policies, and risk management fundamentals     |
| 7     | IT Systems Components           | Roles, operation, and interaction of hardware and software IT components |
| 8     | Basic Scripting and Programming | Creation and execution of scripts and basic programs for automation      |



| KU ID | Knowledge Unit Name                     | Core Competency Description                                                |
|-------|-----------------------------------------|----------------------------------------------------------------------------|
| 9     | Basic Networking                        | Building, operating, and analyzing small-to-medium scale networks          |
| 10    | Network Defense                         | Strategies and tools for protecting networks from cyber threats            |
| 11    | Basic Cryptography                      | Application of cryptographic principles including encryption and hashing   |
| 12    | Operating Systems Concepts              | Roles, architecture, and core services provided by operating systems       |
| 13    | Advanced Network Technology & Protocols | In-depth study of network security protocols and enterprise architecture   |
| 14    | Cloud Computing                         | Cloud service models (IaaS, PaaS, SaaS), deployment, and security          |
| 15    | Cybersecurity Ethics                    | Legal, ethical, and professional responsibilities in cyber contexts        |
| 16    | Database Management Systems             | Administration, optimization, and querying of database systems             |
| 17    | Databases                               | Database usage, management best practices, and common challenges           |
| 18    | Digital Communications                  | Modern digital communication protocols, standards, and infrastructure      |
| 19    | Intrusion Detection / Prevention        | Techniques for detecting, analyzing, and mitigating system vulnerabilities |
| 20    | Low Level Programming                   | Secure programming at the machine and assembly level                       |
| 21    | Network Security Administration         | Design and management of enterprise-scale security infrastructure          |
| 22    | Network Technology and Protocols        | Common networking protocols and their interactions in real environments    |
| 23    | Operating Systems Administration        | System administration tasks, configuration, and maintenance of OSes        |
| 24    | Operating Systems Theory                | Conceptual study of OS components, scheduling, memory, and interfaces      |
| 25    | Vulnerability Analysis                  | Methodologies for discovering and documenting system vulnerabilities       |
| 26    | Web Application Security                | Tools, frameworks, and best practices for securing web applications        |

## 4. Knowledge Unit Matching Engine

### 4.1 Matching Algorithm Design

The matching engine uses Jaccard similarity — a well-established set-theoretic measure — to quantify the semantic overlap between two courses. Because courses are mapped to one or more KUs, the similarity

between two courses can be expressed as the ratio of their shared KUs to the total number of distinct KUs between them.

This approach is intentionally simple and interpretable: advisors and directors can understand exactly why two courses are considered equivalent or non-equivalent by examining which KUs overlap and which do not. This transparency is a deliberate design choice to build trust in the system's recommendations.

#### Jaccard Similarity Formula for KU Matching

Confidence Score =  $| \text{KU}(\text{Course A}) \cap \text{KU}(\text{Course B}) | \div | \text{KU}(\text{Course A}) \cup \text{KU}(\text{Course B}) | \times 100$   
 Worked Example: Course A (CS101 — Data Structures & Algorithms): KU set = { 1, 8, 20 } Course B (CSIA 150 — Computer Science I): KU set = { 1, 8 } Intersection = { 1, 8 }  $\rightarrow$  size = 2 Union = { 1, 8, 20 }  $\rightarrow$  size = 3 Confidence Score =  $2 \div 3 \times 100 = 66.7\% \rightarrow$  Medium Confidence — Director Review Required

## 4.2 Confidence Score Thresholds

Based on the computed confidence score, the system automatically assigns a classification and routes the match accordingly. These thresholds are configurable and will be refined in future phases based on director feedback:

| Score Range | Classification    | System Action                | Director Involvement                         |
|-------------|-------------------|------------------------------|----------------------------------------------|
| 80% – 100%  | High Confidence   | Auto-flagged for approval    | Director confirms — minimal review expected  |
| 50% – 79%   | Medium Confidence | Sent for full manual review  | Director reviews KU evidence and decides     |
| 25% – 49%   | Low Confidence    | Match tentatively rejected   | Director may override with justification     |
| Below 25%   | No Match          | Match rejected automatically | Student notified — no director action needed |

## 4.3 Course-to-KU Mapping — Full Seeded Sample

The many-to-many relationship between courses and Knowledge Units is the engine's core data structure. The 32 seeded course-KU links enable the POC matching pipeline. Courses with more KU associations produce richer, more discriminating matches:

| Course ID | Course Name                  | KU IDs Mapped | KU Count |
|-----------|------------------------------|---------------|----------|
| 1         | Data Structures & Algorithms | KU 1, 8, 20   | 3        |
| 2         | Network Security             | KU 2, 10, 21  | 3        |

| Course ID | Course Name                    | KU IDs Mapped               | KU Count |
|-----------|--------------------------------|-----------------------------|----------|
| 3         | IT Fundamentals                | KU 3, 7                     | 2        |
| 4         | Web Development                | KU 4, 8, 26                 | 3        |
| 5         | Computer Science I (CSIA 150)  | KU 1, 8                     | 2        |
| 12        | Computer Networking (CSIA 301) | KU 9, 13, 18, 22            | 4        |
| 13        | Operating Systems (CSIA 317)   | KU 7, 12, 14, 24            | 4        |
| 17        | Database Systems (CSIA 333)    | KU 16, 17                   | 2        |
| 19        | Cyber Ops (CSIA 319)           | KU 2, 5, 10, 19             | 4        |
| 23        | Intro to Computer Security     | KU 5, 6, 11, 15, 19, 25, 26 | 7        |
| 24        | Internet Security              | KU 10, 21, 24, 26           | 4        |

## 4.4 Matching Limitations and Future Improvements

The Jaccard-based approach in the POC has known limitations that will be addressed in subsequent phases:

- **KU weight equality** — all KUs are treated as equally important. In practice, some KUs (e.g., Cybersecurity Foundations) carry greater academic weight than others (e.g., Basic Cryptography). A weighted Jaccard implementation will improve precision.
- **Binary KU membership** — a course either has a KU or it does not. Future versions will model partial KU coverage (e.g., 70% of a KU's competencies are covered) using rubric-based scoring from syllabus analysis.
- **Credit hour parity** — the current system does not penalize matches where credit hours differ significantly. A 4-credit course matching a 1-credit course should receive a penalty modifier.
- **Human override logging** — director overrides of low-confidence matches are not currently used to retrain the model. Feedback loops will be added in Phase 2.

## 5. PDF Transcript Parser

### 5.1 Purpose and Motivation

Student academic transcripts are the primary source of truth for transfer credit evaluation. In practice, transcripts arrive in two common formats: Banner-generated PDFs (the most structured format, produced directly by institutional SIS systems) and web-saved or printed PDFs, which often contain layout artifacts, irregular whitespace, and merged table cells.

The `pdf_reader.py` script automates extraction of course records from both formats, eliminating the need for students to manually re-enter their academic history and reducing data entry errors that could corrupt the matching pipeline.

## 5.2 Technical Approach

The parser uses a layered extraction strategy: it first attempts table-based extraction using pdfplumber, which reliably handles structured Banner transcripts. If table extraction yields insufficient results (fewer than a threshold number of course rows), the parser falls back to regex-based line-by-line parsing to handle unstructured layouts:

```
# pdf_reader.py - Simplified extraction flow

import pdfplumber, re, pandas as pd

def parse_transcript(pdf_path: str) -> pd.DataFrame:
    records = []

    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            # Attempt 1: structured table extraction (Banner PDFs)
            tables = page.extract_tables()
            for table in tables:
                for row in table:
                    if is_course_row(row):
                        records.append(parse_row(row))

            # Attempt 2: regex fallback (web-saved / noisy PDFs)
            if len(records) < MIN_EXPECTED_ROWS:
                text = page.extract_text() or ''
                pattern = r"([A-Z]{2,6}s*\d{3,4})\s+(.+?)\s+([A-F][+-]
]?)\s+(\d+\.\d*)"
                for match in re.finditer(pattern, text):
                    records.append({
                        "course_code" : match.group(1).strip(),
                        "course_title" : match.group(2).strip(),
                        "grade" : match.group(3).strip(),
                        "credits" : float(match.group(4))
                    })

    return pd.DataFrame(records).drop_duplicates()
```

## 5.3 Extracted Fields and Output Schema

| Field        | Type   | Source                     | Required | Notes                       |
|--------------|--------|----------------------------|----------|-----------------------------|
| course_code  | String | Table cell / Regex group 1 | Yes      | e.g., CSIA 150, CS101       |
| course_title | String | Table cell / Regex group 2 | Yes      | Full course name as printed |
| grade        | String | Table cell / Regex group 3 | No       | Letter grade; W/I handled   |
| credits      | Float  | Table cell / Regex group 4 | Yes      | Credit hours as decimal     |

## 5.4 Libraries and Dependencies

| Library    | Version | Purpose                                                         |
|------------|---------|-----------------------------------------------------------------|
| pdfplumber | ≥ 0.9   | Primary table-based extraction for structured Banner PDFs       |
| re         | stdlib  | Regex-based fallback parsing for unstructured PDF layouts       |
| pandas     | ≥ 2.0   | Structuring, deduplicating, and exporting extracted course data |

## 5.5 Usage

```
# Basic usage
python pdf_reader.py --pdf ./transcripts/student_001.pdf

# Output to CSV
python pdf_reader.py --pdf ./transcripts/student_001.pdf --output
courses_extracted.csv

# Verbose mode (shows extraction method used per page)
python pdf_reader.py --pdf ./transcripts/student_001.pdf --verbose
```

# 6. Data Standardization and Validation Layer

## 6.1 Purpose

Even a well-implemented parser will produce imperfect output when source data is noisy. This layer acts as a quality gate between the parser output and the KU matching engine, ensuring that only clean, well-formed course records enter the matching pipeline. Rows that cannot be fully standardized are held for manual review rather than silently passed through with missing data.

## 6.2 Standardization Pipeline

The standardization layer processes parsed course data through three sequential steps:

1. Field validation — check that `course_code`, `course_title`, and `credits` are all present and non-null.
2. Value normalization — convert course codes to uppercase, strip whitespace, normalize credit values to one decimal place.

3. KU terminology normalization — map variant KU label strings from parser output to canonical internal KU names.

## 6.3 KU Normalization — Mapping Variant Labels

Because course syllabi and instructor notes may describe competencies in varying language, the normalization layer maintains a curated mapping dictionary that resolves synonyms and variants to a single canonical KU label:

```
# KU normalization mapping — standardization/ku_normalizer.py

KU_MAP = {
    # Binary / low-level programming variants
    "binary arithmetic"      : "KU_LOW_LEVEL_PROGRAMMING",
    "base conversion"        : "KU_LOW_LEVEL_PROGRAMMING",
    "assembly language"      : "KU_LOW_LEVEL_PROGRAMMING",
    "machine code"           : "KU_LOW_LEVEL_PROGRAMMING",
    "computer organization basics" : "KU_IT_SYSTEMS_COMPONENTS",
    # Networking variants
    "tcp/ip fundamentals"    : "KU_NETWORKING_FUNDAMENTALS",
    "osi model"               : "KU_NETWORKING_FUNDAMENTALS",
    "packet switching"        : "KU_BASIC_NETWORKING",
    # Security variants
    "intro to cybersecurity"  : "KU_CYBERSECURITY_BASICS",
    "information security basics" : "KU_CYBERSECURITY_BASICS",
    "pen testing fundamentals" : "KU_VULNERABILITY_ANALYSIS",
}

def normalize_kus(raw_kus: list[str]) -> list[str]:
    return [KU_MAP.get(k.lower().strip(), k) for k in raw_kus]
```

## 6.4 Parser Output Validation

Before any parsed course record is inserted into the transfer request pipeline, it is validated against a required-field schema. Records with missing critical fields are written to a separate `flagged_records` table for advisor follow-up:

```
import pandas as pd

REQUIRED_FIELDS = ['course_code', 'course_title', 'credits']

def validate_parsed_records(df: pd.DataFrame) -> tuple[pd.DataFrame,
pd.DataFrame]:
    """Returns (clean_df, flagged_df)."""
    flag_mask = df[REQUIRED_FIELDS].isna().any(axis=1)
    clean_df = df[~flag_mask].copy()
    flagged_df = df[flag_mask].copy()
```

```
# Normalize clean records
clean_df['course_code'] = clean_df['course_code'].str.upper().str.strip()
clean_df['course_title'] =
clean_df['course_title'].str.strip().str.title()
clean_df['credits'] = clean_df['credits'].round(1)

print(f'  Clean records   : {len(clean_df)}')
print(f'  Flagged records: {len(flagged_df)}')
return clean_df, flagged_df
```

## 6.5 Benchmark Dataset Generation

To support testing of the matching pipeline before large volumes of real student data are available, the standardization layer includes utilities for generating synthetic transcript datasets. Synthetic records are parameterized to cover common edge cases such as courses with many KUs, courses with no KUs mapped, and courses with matching credit hours but divergent content.

### Value of Benchmark Datasets

Synthetic benchmarks allow the team to: (1) verify that the confidence scoring formula produces correct outputs across the full 0–100% range, (2) test that threshold routing (High / Medium / Low / No Match) operates as expected, and (3) measure the precision and recall of the matching engine before live student data is available. Benchmark records are clearly tagged to prevent them from appearing in production match results.

## 7. End-to-End Transfer Request Example

### 7.1 Scenario Description

The following walkthrough traces a single transfer request through every layer of the TCM pipeline — from student transcript upload through director decision and audit logging. This is the example seeded in the POC database.

### 7.2 Student and Enrollment Context

|             |                                          |
|-------------|------------------------------------------|
| Student ID  | 1                                        |
| Enrolled At | Roosevelt University (Institution ID: 1) |
| Program     | Computer Science (Program ID: 1)         |

## 7.3 Transfer Request Details

| Field                   | Value                                                                    |
|-------------------------|--------------------------------------------------------------------------|
| Request ID              | 1                                                                        |
| Submission Date         | 2026-02-16                                                               |
| Source Institution      | Roosevelt University (Institution ID: 1)                                 |
| Destination Institution | Harold Washington College (Institution ID: 4)                            |
| Source Course           | CS101 — Data Structures and Algorithms (4 credits, Roosevelt University) |
| Target Course           | SD102 — Web Development (3 credits, Harold Washington College)           |
| Request Status          | Pending — Awaiting Director Review                                       |

## 7.4 Matching Engine Processing

When the system processes this transfer request, it executes the following steps:

4. Retrieve KU profile for CS101 (Data Structures and Algorithms): { KU 1 — Programming Concepts, KU 8 — Basic Scripting and Programming, KU 20 — Low Level Programming }
5. Retrieve KU profile for SD102 (Web Development): { KU 4 — Database Management, KU 8 — Basic Scripting and Programming, KU 26 — Web Application Security }
6. Calculate Jaccard similarity: Shared KUs = { KU 8 } = 1 shared. Union = { KU 1, 4, 8, 20, 26 } = 5 total. Confidence Score =  $1 \div 5 \times 100 = 20\%$ .
7. Apply threshold: 20% falls below the 25% minimum — classified as No Match. Transfer request is rejected at this stage.
8. Director notification: Program director at Harold Washington College is notified. Director may review and override with written justification.
9. Audit log: All processing steps, timestamps, and the final status are written to match\_history.

### Design Validation Note

This example intentionally demonstrates a No Match result. CS101 (Data Structures) and SD102 (Web Development) share only one KU out of five, correctly reflecting that these are substantively different courses. The system correctly prevents inappropriate auto-approval. The POC seeding includes this case specifically to validate threshold behavior at the low end of the confidence scale.



## 8. Director Review Workflow

### 8.1 Overview

The director review workflow is the human governance layer of the TCM system. All match records — regardless of confidence score — are visible to program directors at the destination institution. High-confidence matches require only a brief confirmation; medium-confidence matches require the director to examine KU overlap evidence; low-confidence and no-match results require an explicit override decision with justification if the director disagrees with the system's recommendation.

### 8.2 Review State Machine

| Status    | Set By   | Condition                           | Next States                 | Description                         |
|-----------|----------|-------------------------------------|-----------------------------|-------------------------------------|
| Pending   | System   | Match record created                | Approved, Denied, Escalated | Awaiting director review            |
| Approved  | Director | Director accepts the match          | Closed                      | Transfer credit granted             |
| Denied    | Director | Director rejects the match          | Closed                      | Student notified; no credit awarded |
| Escalated | Director | Director defers to department chair | Approved, Denied            | Chair makes final decision          |
| Closed    | System   | Final decision recorded             | — (terminal)                | Audit record finalized              |

### 8.3 What Directors See

When a director opens a pending match for review, they are presented with the following information to support their decision:

- The full KU profile of the source course — name, code, institution, and all mapped KUs with descriptions.
- The full KU profile of the target course — same detail level.
- A visual comparison showing shared KUs (matched), KUs only in the source (unmatched in target), and KUs only in the target (unmatched in source).
- The computed confidence score and its classification (High / Medium / Low / No Match).
- The student's grade in the source course and the credit hour differential between source and target.
- Historical decisions for the same course pair, if any prior decisions exist in `match_history`.

### 8.4 Audit Trail and Compliance

Every action in the review workflow is written to the `match_history` table as an immutable record. The audit trail captures the actor (director ID), the action taken (approved / denied / escalated), a timestamp, and any

free-text justification entered by the director. This log is append-only and cannot be modified after creation, ensuring the integrity of the institutional record for accreditation and compliance purposes.

## 9. Future Enhancements

### 9.1 Phase 2 Roadmap

The following enhancements are planned for the next development phase following successful POC validation. Each enhancement is prioritized by impact on system accuracy and user experience:

| Priority | Enhancement                      | Rationale                                                                                                                |
|----------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| High     | Weighted KU Scoring              | Some KUs carry more academic weight. A weighted Jaccard model will improve match precision.                              |
| High     | OCR / Scanned Transcript Support | Many students only have image-based transcripts. OCR integration with Tesseract or AWS Textract will eliminate this gap. |
| High     | Director Review Web Dashboard    | A browser-based UI will replace the current data-model-only director workflow, enabling real-world testing.              |
| Medium   | Expanded Institution Coverage    | Onboarding additional Chicago-area and Illinois colleges will validate the KU taxonomy at scale.                         |
| Medium   | Student-Facing Portal            | Self-service transcript upload and real-time match status tracking will reduce advisor workload.                         |
| Medium   | Feedback Loop / Model Tuning     | Director overrides will feed back into confidence threshold calibration for continuous improvement.                      |
| Low      | API Layer                        | A REST API will allow integration with institutional SIS systems for automated transfer request submission.              |
| Low      | Reporting and Analytics          | Aggregate match rate, approval rate, and KU coverage dashboards for institutional decision-making.                       |

### 9.2 Scalability Considerations

The current POC operates on 25 courses, 26 KUs, and 32 course-KU links across 4 institutions. Scaling to hundreds of courses across dozens of institutions will require:

- Database indexing on course\_ku join table to maintain sub-second query performance for KU profile lookups.
- Caching of KU profiles for frequently queried courses to reduce repeated database reads.
- Batch processing mode for the PDF parser to handle bulk transcript uploads during registration periods.

- Role-based access control (RBAC) to ensure directors can only review matches for their own institution's programs.

## 10. Contributors — Data Science Branch

### 10.1 Team Contributions

The following contributions were delivered by the Data Science team for the TCM proof of concept:

| Contribution Area     | Deliverable                                               | Status   |
|-----------------------|-----------------------------------------------------------|----------|
| Data Modeling         | Entity-relationship design; 11 CSV seed files             | Complete |
| Transcript Parsing    | pdf_reader.py — Banner & web-saved PDF extraction         | Complete |
| KU Taxonomy Design    | 26 CAE-aligned Knowledge Units with descriptions          | Complete |
| Course-KU Mapping     | 32 course-to-KU links across all seeded programs          | Complete |
| Standardization Layer | KU normalization map; parser validation utilities         | Complete |
| Benchmark Datasets    | Synthetic test cases covering full confidence score range | Complete |
| Documentation         | This technical proof of concept documentation             | Complete |

## 11. Glossary

| Term               | Definition                                                                                                                                                         |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Banner             | A widely used Student Information System (SIS) by Ellucian. Generates structured PDF transcripts used as primary parser input.                                     |
| CAE                | Centers of Academic Excellence — an NSA/DHS designation awarded to institutions whose cybersecurity programs meet nationally standardized curriculum requirements. |
| Confidence Score   | A percentage value (0–100%) computed using Jaccard similarity, representing the degree of KU overlap between two courses from different institutions.              |
| Course Match       | A database record pairing a source course with a target course, generated by the KU matching engine and awaiting director review.                                  |
| Jaccard Similarity | A set-theoretic similarity measure: $ A \cap B  /  A \cup B $ . Applied here to compare Knowledge Unit sets between two courses.                                   |

| Term             | Definition                                                                                                                                          |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| KU               | Knowledge Unit — a discrete, standardized academic competency area used to semantically describe and compare course content across institutions.    |
| Match History    | The append-only audit log table that records every transfer credit decision, including the actor, timestamp, and justification.                     |
| POC              | Proof of Concept — the initial development phase demonstrating core system functionality with limited seed data before full-scale deployment.       |
| SIS              | Student Information System — the administrative software institutions use to manage enrollment, grades, and transcripts (e.g., Banner, PeopleSoft). |
| Transfer Request | A student-submitted request to have a completed course at one institution recognized as satisfying a requirement at the destination institution.    |

---

*Transfer Credit Match System · Data Science Branch · Proof of Concept · April 2026*